# Diffusion Language Models are Faster than Autoregressive on CPU: An Empirical Study of the Memory-Compute Regime Inversion

Carmen Esteban*
IAFiscal & Partners, Madrid, Spain

March 22, 2026

**Abstract**

Diffusion language models (dLLMs) are widely considered slower than autoregressive (AR) models for inference. We demonstrate empirically that this conclusion is hardware-dependent: on commodity CPUs, dLLMs can outperform optimized AR inference. We identify the mechanism—a regime inversion from memory-bound (AR) to compute-bound (diffusion)—and validate it with systematic benchmarks. We present `diffuse-cpp`, the first C++ inference engine for diffusion LLMs with GGML quantization support. On a 12-core AMD EPYC, `diffuse-cpp` supports two models—LLaDA-8B (Llama backbone, MHA) and Dream-7B (Qwen2.5 backbone, GQA)—and achieves 14–28 tok/s with entropy-exit scheduling, inter-step KV cache, and 256-token generation on easy prompts, up to 3.3× faster than `llama.cpp` (8.51 tok/s) on the same hardware. Dream excels at math and code (21.6 tok/s, correctly solving arithmetic), while LLaDA excels at translation (27.7 tok/s). Thread scaling reaches 7.5× from 1 to 12 cores for diffusion versus saturation at 4 cores for AR. An entropy-based adaptive scheduler reduces denoising steps from 16 to 2–4 on easy prompts without quality loss. An inter-step KV cache reuses Key and Value tensors between denoising steps, providing 1.6× average speedup with implicit regularization benefits. These results suggest that diffusion models may be a viable path toward democratizing LLM inference on commodity hardware without GPU dependency.

**Keywords:** diffusion language models, CPU inference, memory-bound, compute-bound, LLM acceleration, quantization, adaptive scheduling, inter-step cache, multi-model

**License:** CC-BY-4.0 (paper), AGPL-3.0 (code)

## 1 Introduction

Large language model (LLM) inference is dominated by GPU hardware. The supply chain—from ASML's lithography monopoly through TSMC's fabrication to NVIDIA's tensor cores—creates a dependency that concentrates access to AI capabilities among well-funded actors. The fundamental question motivating this work is whether alternative computational paradigms can make LLM inference viable on commodity CPUs, which are ubiquitous and inexpensive.

Autoregressive (AR) models generate text one token at a time. Each token requires a full pass through the model's weight matrix, reading billions of parameters from memory to produce a single output. On CPUs, where memory bandwidth is 10–50× lower than on GPUs, this sequential pattern creates a severe bottleneck: the processor spends most of its time waiting for data rather

*Corresponding author: caresment@gmail.com. ORCID: 0009-0009-8062-5492

than computing. Adding more CPU cores does not help, because all threads compete for the same memory bus. This is the memory-bound regime.

Diffusion language models (dLLMs) [7, 6, 8] operate differently. Instead of generating tokens sequentially, they begin with a fully masked sequence and iteratively refine all positions in parallel across $T$ denoising steps. Each step reads the model weights once but processes $B$ tokens simultaneously through matrix-matrix multiplication rather than matrix-vector. This amortizes the cost of memory access across many tokens, shifting the workload from memory-bound to compute-bound. Crucially, matrix-matrix multiplication parallelizes efficiently across CPU cores.

Despite this favorable computational profile, the dLLM literature has focused exclusively on GPU benchmarks. Papers introducing LLaDA [7], SEDD [6], and MDLM [8] report wall-clock times on A100 and H100 GPUs, where the massive memory bandwidth makes AR models fast and the compute advantages of diffusion are less pronounced. No prior work has measured dLLM inference on commodity CPUs.

This paper makes six contributions:

1. **First empirical demonstration** of the memory-compute regime inversion on CPU: we show that dLLMs achieve near-linear thread scaling (7.5× at 12 cores) while AR inference saturates at 4 cores (2.4×).

2. `diffuse-cpp`: the first C++ inference engine for dLLMs, built on GGML [2], supporting two model architectures: LLaDA-8B (Llama backbone, MHA) and Dream-7B (Qwen2.5 backbone, GQA). Both use F16, Q8_0, and Q4_K_M quantization.

3. **Entropy-exit adaptive scheduling**: an entropy-based early termination mechanism that reduces denoising steps from 16 to 2–4 on easy prompts, providing up to 6× speedup over the F16 baseline with no quality degradation.

4. **Inter-step KV cache**: a caching mechanism that reuses Key and Value tensors between denoising steps, reducing per-step cost by only recomputing active positions. Achieves 1.6× average speedup with an implicit regularization effect that improves output quality on 3 of 8 benchmarked prompts.

5. **Multi-model comparative benchmarking**: systematic comparison of two dLLM architectures (MHA vs GQA) on identical hardware, revealing complementary strengths—Dream-7B excels at math and code while LLaDA-8B excels at translation.

6. **Systematic benchmarking**: 288 configurations with 3 repetitions each, plus 8 diverse real prompts across 2 models, totaling over 8 hours of continuous CPU measurement.

# 2 Background

## 2.1 Autoregressive Inference

AR models generate tokens sequentially: at each step $t$, the model computes $\mathbf{y}_t = f(\mathbf{x}_{1:t})$ by multiplying the input embedding against the weight matrices. For a single new token, this reduces to a matrix-vector product $\mathbf{W} \cdot \mathbf{h}$ where $\mathbf{W} \in \mathbb{R}^{N \times K}$ and $\mathbf{h} \in \mathbb{R}^K$. The arithmetic intensity (AI) is:

$$\mathrm{AI}_{\mathrm{AR}} = \frac{2NK}{(N+K) \cdot b} \approx \frac{2}{b} \quad \text{FLOP/byte} \tag{1}$$

where $b$ is the bytes per parameter ($b = 2$ for FP16, $b \approx 0.5$ for Q4_K_M). For Q4_K_M, $\text{AI}_{\text{AR}} \approx 4$ FLOP/byte. KV caching eliminates redundant computation but does not change the fundamental memory access pattern.

### 2.2 Diffusion Inference

Diffusion LLMs start with a fully masked sequence of $B$ tokens and refine it over $T$ steps. Each step performs a full forward pass on the entire sequence, producing matrix-matrix products $\mathbf{W} \cdot \mathbf{H}$ where $\mathbf{H} \in \mathbb{R}^{K \times B}$. The arithmetic intensity is:

$$\text{AI}_{\text{diff}} = \frac{2NKB}{(NK + KB) \cdot b} \approx \frac{2B}{b} \quad \text{FLOP/byte} \tag{2}$$

For $B = 64$ and Q4_K_M: $\text{AI}_{\text{diff}} \approx 256$ FLOP/byte; for $B = 256$: $\text{AI}_{\text{diff}} \approx 1024$ FLOP/byte—two to three orders of magnitude higher than AR. Crucially, larger $B$ increases AI without adding memory traffic, so longer generation buffers make diffusion *more* compute-efficient per token (Section 6.6). There is no KV cache; each step recomputes attention over the full sequence with bidirectional (non-causal) masking.

### 2.3 Prior Work

LLaDA [7] introduced masked discrete diffusion for language modeling with a Llama-style backbone and Multi-Head Attention (MHA). Dream [12] adopted a Qwen2.5 backbone with Grouped Query Attention (GQA), autoregressive logit shift for causal bias, and additional remasking strategies. SEDD [6] proposed score-based discrete diffusion. MDLM [8] simplified the training objective. Prophet [5] introduced speculative decoding for discrete diffusion, using a confidence-gap criterion for early stopping. EAGS [4] and Fast-dLLM [11] explored entropy-adaptive scheduling. All prior work benchmarked on GPU.

For CPU inference, `llama.cpp` [3] is the state of the art for AR models, achieving near-theoretical throughput through GGML's optimized SIMD kernels and quantization formats.

## 3 The Regime Inversion

The central insight of this paper is that the relative performance of AR and diffusion inference *inverts* depending on the hardware's memory-bandwidth-to-compute ratio. We formalize this using the roofline model [10].

### 3.1 The Matching Argument

A processor achieves peak performance when the workload's arithmetic intensity matches the machine's operational intensity (peak FLOPS / peak bandwidth). Table 1 shows the matching:

Table 1: Memory-compute matching across hardware regimes.

| | **GPU** (high BW) | **CPU** (low BW) |
|---|---|---|
| **AR** (BW-hungry) | Match → fast | Mismatch → slow |
| **Diffusion** (compute-hungry) | Redundant | Match → scales |

On GPUs with ∼1 TB/s memory bandwidth, AR's low arithmetic intensity is tolerable: the hardware delivers data fast enough. Diffusion's high compute demand, while achievable, offers no advantage over highly optimized AR inference with KV caching.

On CPUs with ∼50–80 GB/s bandwidth, the situation reverses. AR inference hits a hard ceiling: with a 5.1 GB model (Q4_K_M) and 75 GB/s bandwidth, the theoretical maximum is $75/5.1 \approx 14.7$ tokens/s, and practical throughput is 8.51 tok/s (58% of theoretical). Adding threads provides diminishing returns because all cores share the memory bus.

Diffusion inference, in contrast, is compute-bound on CPU. Each forward pass performs $\mathcal{O}(NKB)$ operations with $\mathcal{O}(NK)$ data movement, and matrix-matrix multiplication parallelizes efficiently across cores via BLAS routines. The throughput scales with available compute, not bandwidth.

### 3.2 Empirical Evidence: Thread Scaling

Figure **??** shows the critical empirical evidence. We benchmark both `diffuse-cpp` (LLaDA-8B Q4_K_M, $T$=16 steps) and `llama.cpp` (Llama-3-8B Q4_K_M) on identical hardware (AMD EPYC 4465P, 12 physical cores).

The AR curve saturates at 4 threads (scaling from 1.0× to 2.5×, then stalling). The diffusion curve continues to 7.4× at 12 threads, with degradation only at 24 threads due to hyperthreading contention. This demonstrates compute-bound scaling for diffusion: each additional core contributes proportionally to throughput, as predicted by the roofline analysis.

### 3.3 The Crossover

In absolute terms, `llama.cpp` remains faster for raw token generation at 12 cores (8.51 tok/s for 16-step diffusion). However, this comparison is misleading for two reasons. First, entropy-exit scheduling (Section 5) reduces effective steps to 2–4 on favorable prompts, pushing diffusion throughput to 15–28 tok/s and surpassing AR. Second, the scaling trends diverge: as core counts increase (modern server CPUs have 64–128 cores), diffusion throughput grows near-linearly while AR remains bandwidth-limited. The crossover for unconditional throughput likely occurs between 16 and 32 cores on current hardware.

## 4 `diffuse-cpp`: Architecture

`diffuse-cpp` is a C++ inference engine for masked diffusion language models, built on the GGML tensor library [2]—the same foundation as `llama.cpp`. This section describes its design.

**Supported architectures.** `diffuse-cpp` supports two model families:

- **LLaDA-8B** [7]: Llama backbone, 32 layers, 4096 hidden dim, Multi-Head Attention (32 query / 32 KV heads), SwiGLU FFN (12288 intermediate), vocabulary 126,464, RoPE $\theta$=500,000.
- **Dream-7B** [12]: Qwen2.5 backbone, 28 layers, 3584 hidden dim, Grouped Query Attention (28 query / 4 KV heads), SwiGLU FFN (18944 intermediate), vocabulary 152,064, RoPE $\theta$=1,000,000. Dream adds QKV biases (kept at F32 in all quantizations), an autoregressive logit shift for causal bias, and additional remasking strategies (`maskgit_plus`, `topk_margin`).

Both models use bidirectional (non-causal) attention. The shared forward pass differs only in GQA handling: for Dream, each KV head is expanded to serve $n_{\text{rep}} = 7$ query heads via grouped repetition before attention computation.

**Denoising loop.** Generation proceeds as follows: (1) initialize a sequence of $B$ mask tokens; (2) perform a forward pass to obtain logits for all positions; (3) for each masked position, sample from the logit distribution; (4) unmask positions based on the scheduling policy; (5) repeat until all positions are unmasked or the step budget is exhausted.

**Model format.** Models are stored in GGUF format, converted from HuggingFace SafeTensors via dedicated Python scripts (`tools/convert-llada.py` and `tools/convert-dream.py`). LLaDA produces 226 tensors; Dream produces 339 tensors (255 weights + 84 QKV biases).

**Quantization.** `diffuse-cpp` supports three quantization levels via `diffuse-quantize`: F16, Q8_0, and Q4_K_M. For LLaDA: 14.9/8.4/5.1 GB; for Dream: 15.2/8.6/5.3 GB. Quantization is weight-only; activations remain in FP32. QKV biases (Dream only) are preserved at F32 in all quantization levels.

**Scheduling policies.** Two scheduling policies are implemented. `low_confidence` is the default baseline: at each step, it unmasks a fixed fraction of the most confident positions (lowest logit entropy). `entropy_exit` is our adaptive scheduler, described in Section 5.

### 4.1 Inter-Step KV Cache

Standard diffusion inference recomputes the full attention for all $N$ positions at every denoising step, even though most positions do not change between consecutive steps. Once a position is unmasked with high confidence, its token value—and therefore its Key and Value vectors—remains stable. We exploit this by caching the post-RoPE K and V tensors from each transformer layer between steps.

**Mechanism.** After each full forward pass, we store the K and V tensors for all $L$ layers and all $N$ positions. On subsequent steps, we partition positions into two sets:

- **Active set** $\mathcal{A}$: positions that are still masked or were recently unmasked. These require fresh K,V computation because their hidden states may have changed.
- **Cached set** $\mathcal{C} = \{1, \ldots, N\} \setminus \mathcal{A}$: positions whose tokens have not changed. Their K,V are reused from the cache.

For each layer, the attention computation becomes:

$$\mathrm{Attn}(\mathbf{Q}_{\mathcal{A}}, [\mathbf{K}_{\mathcal{C}}^{\mathrm{cached}} \| \mathbf{K}_{\mathcal{A}}^{\mathrm{fresh}}], [\mathbf{V}_{\mathcal{C}}^{\mathrm{cached}} \| \mathbf{V}_{\mathcal{A}}^{\mathrm{fresh}}])$$

where $\|$ denotes concatenation along the sequence dimension.

**Complexity.** The per-step cost reduces from $\mathcal{O}(N^2)$ per layer (full recomputation) to $\mathcal{O}(|\mathcal{A}| \cdot N)$ per layer (only active queries, full key-value context). Since $|\mathcal{A}|$ shrinks monotonically as tokens converge, later steps become progressively cheaper.

**Approximation.** The cache introduces an approximation: the K,V for cached positions are computed from *stale* hidden states. In a bidirectional transformer, changing any token affects all positions through attention interactions. However, we observe empirically that this approximation does not degrade output quality. In fact, it acts as implicit regularization: reusing stable K,V from earlier steps prevents attention from amplifying accumulated errors during the denoising process. On 3 of 8 benchmarked prompts, cached output quality is measurably better than uncached (Section 6.7).

**Configuration.** The cache is enabled by default. The flag `--no-cache` disables it. The flag `--cache-keep-active N` extends the active set to include positions that changed within the last $N$ steps, trading speed for fidelity.

# 5 Entropy-Exit Adaptive Scheduling

Standard diffusion schedulers use a fixed number of denoising steps $T$ regardless of prompt difficulty. This is suboptimal: a factual question ("What is the capital of France?") requires far less refinement than creative writing ("Write a poem about the ocean"). We introduce `entropy_exit`, an adaptive scheduler that allows the model to decide when to stop.

## 5.1 Algorithm

After each denoising step, `entropy_exit` computes the Shannon entropy [9] of the logit distribution at each remaining masked position:

$$H_i = -\sum_{v=1}^{V} p_{i,v} \log p_{i,v} \tag{3}$$

where $p_{i,v} = \text{softmax}(\mathbf{z}_i)_v$ and $V$ is the vocabulary size. Positions with $H_i < \tau$ (default $\tau = 1.5$ nats) are immediately unmasked—the model is sufficiently confident. If all $B$ positions are unmasked, generation terminates.

The computational overhead is negligible: entropy computation is $\mathcal{O}(BV)$, while the forward pass is $\mathcal{O}(BNK)$ with $N, K \gg V$ for the dominant attention and MLP layers.

## 5.2 Prior Art

Prophet [5] uses a confidence-gap criterion (difference between top-2 logits) for speculative decoding of diffusion models. EAGS [4] and Fast-dLLM [11] explore entropy-based scheduling. Our contribution is not the scheduling concept itself, but its implementation in a CPU-optimized engine where the memory-compute regime inversion makes adaptive scheduling particularly impactful: reducing steps has a direct, near-linear effect on throughput in the compute-bound regime.

## 5.3 Empirical Behavior

Easy prompts (factual QA, arithmetic, simple translation) converge in 2–4 steps, achieving 15–28 tok/s with inter-step cache. Medium-difficulty prompts (code generation, complex translation) use 7–15 steps. Hard prompts (creative writing, open-ended explanation) use the full 15–16 steps with no speedup but also no degradation relative to the baseline.

In a separate PyTorch-based evaluation across 42 prompts in 12 categories, 55% of prompts achieved >20% speedup. The best categories were instruction-following (3.86×), classification (2.84×), and translation (2.71×). No category showed degradation.

# 6 Experimental Results

## 6.1 Setup

All benchmarks were conducted on a dedicated AMD EPYC 4465P (12 cores, 24 threads), 125 GB DDR5 RAM, running Ubuntu 24.04 with GCC 13.3. No competing processes ran during measurement. Each configuration was run 3 times with 1 warmup, generating 64 tokens from a 32-token prompt unless otherwise noted. Sequence-length scaling experiments (Section 6.6) used 256 generated tokens. GGML compute buffers used a 1.5× safety multiplier per upstream recommendation.

**Methodology note.** Initial benchmarks used a 1.2× buffer multiplier, yielding approximately 30% higher throughput (e.g., 13.59 tok/s peak). Upon discovering this was below the GGML-recommended minimum, all 288 configurations were re-run with 1.5× buffers. This paper reports only the corrected results.

### 6.2 Quantization Performance

Table 2 shows the effect of quantization at a fixed configuration ($T$=16 steps, 12 threads).

Table 2: Performance by quantization level (steps=16, threads=12, LLaDA-8B).

| Model | Size | low_confidence | Per-step | vs F16 |
|---|---|---|---|---|
| F16 | 14.9 GB | 1.64 tok/s | 2433 ms | 1.00× |
| Q8_0 | 8.4 GB | 1.84 tok/s | 2182 ms | 1.12× |
| Q4_K_M | 5.1 GB | 2.52 tok/s | 1586 ms | 1.54× |

The 1.54× speedup from F16 to Q4_K_M is lower than the 2.9× size reduction would suggest. This is consistent with compute-bound behavior: reducing data size helps (fewer bytes to dequantize), but the bottleneck is the matrix multiplication itself, not the memory transfer.

### 6.3 Thread Scaling

Table 3 shows thread scaling for Q4_K_M at 16 steps.

Table 3: Thread scaling (Q4_K_M, steps=16, `low_confidence`).

| Threads | tok/s | Scaling |
|---|---|---|
| 1 | 0.34 | 1.0× |
| 4 | 1.18 | 3.5× |
| 12 | 2.52 | 7.4× |
| 24 | 2.21 | 6.5× |

Scaling is near-linear up to the physical core count (12), confirming compute-bound behavior. Hyperthreading (24 threads) causes a 12% regression due to shared execution units. Optimal thread count equals the number of physical cores.

### 6.4 Real-Prompt Performance

Table 4 shows end-to-end `entropy_exit` performance on 8 diverse prompts using Q4_K_M, $T$=16, 12 threads, at both $B$=64 and $B$=256, with and without inter-step cache.

The pattern at $B$=64 is clear: easy prompts (factual, arithmetic, simple translation) converge in 2–4 steps with 4–6× speedup over `low_confidence`. Hard prompts use full steps with no speedup but no degradation.

At $B$=256, all prompts benefit from the larger generation buffer, with throughput gains of 23–113% on 7 of 8 prompts. With inter-step cache enabled (default), throughput increases by an additional 1.59× on average, with the highest gains on prompts that require many steps (up to 3.66× on 16-step prompts).

Table 4: Real-prompt `entropy_exit` performance (Q4_K_M, steps=16, threads=12).

| | $B$=64 | | $B$=256 | | |
|---|---|---|---|---|---|
| **Prompt** | **tok/s** | **Steps** | **No-Cache** | **Cache** | **Steps** |
| Capital of France? | 9.22 | 4 | 17.5 | **24.4** | 3 |
| Translate to French | 10.23 | 3 | 25.9 | **27.7** | 2 |
| $15 \times 23 =$? | 11.49 | 3 | 12.8 | **15.7** | 4 |
| Translate to Spanish | 4.59 | 8 | 7.6 | **22.9** | 7 |
| Python `is_prime` | 2.53 | 15 | 3.2 | **4.9** | 16 |
| Poem about ocean | 2.33 | 17 | 3.2 | **5.3** | 16 |
| Why sky is blue | 2.21 | 17 | 3.3 | **12.0** | 16 |
| List the planets | 2.33 | 17 | 3.3 | **9.4** | 15 |

### 6.5 Comparison with `llama.cpp`

Table 5 compares `diffuse-cpp` with `llama.cpp` on the same hardware. We emphasize that this compares different models (LLaDA-8B vs Llama-3-8B) and different paradigms, not identical systems.

Table 5: Comparison with `llama.cpp` (both Q4_K_M, 12 threads, same hardware).

| **Scenario** | **diffuse-cpp** ($B$=256, cache) | **llama.cpp** |
|---|---|---|
| Easy prompts (EE) | **15–28** tok/s | 8.51 tok/s |
| Medium prompts (EE) | 5–23 tok/s | 8.51 tok/s |
| Hard prompts (EE) | 5–12 tok/s | 8.51 tok/s |
| Thread scaling ($1 \rightarrow 12$) | 7.4× | 2.4× |

With $B$=256 and inter-step cache, `diffuse-cpp` outperforms `llama.cpp` on 6 of 8 prompts (up to 3.3× on French translation). Two prompts (code generation, creative writing) remain slower due to requiring all 16 denoising steps. The thread scaling trends further favor diffusion on many-core machines.

### 6.6 Sequence Length Scaling

Table 6 shows the effect of increasing the generation buffer from 64 to 256 tokens (Q4_K_M, $T$=16 steps, 12 threads, real chat-formatted prompt).

Table 6: Sequence length scaling (Q4_K_M, steps=16, threads=12, 32-token prompt).

| **Scheduler** | $B$=64 | $B$=256 | **Time ratio** | **ms/token** | **Improvement** |
|---|---|---|---|---|---|
| `low_confidence` | 2.58 tok/s | 3.18 tok/s | 3.24× | $388 \rightarrow 314$ | −19% |
| `entropy_exit` | 13.4 tok/s | 17.1 tok/s | 3.15× | $74.5 \rightarrow 58.6$ | −21% |

Generating 4× more tokens increases total wall-clock time by only 3.2×, yielding a 20% reduction in per-token cost. This sub-linear scaling confirms the theoretical prediction from Equation 2: larger $B$ increases arithmetic intensity, amortizing fixed overhead (weight loading, softmax over vocabulary, memory allocation) across more useful tokens.

The practical implication is significant: `entropy_exit` with $B$=256 reaches **15–28 tok/s** on real prompts (Table 4)—up to 3.3× the throughput of `llama.cpp` (8.51 tok/s) on the same hardware. This is the configuration where diffusion's matrix-matrix advantage becomes most pronounced, and it corresponds to the sequence lengths users actually need in production (responses of 200+ tokens are typical for chatbot, code generation, and translation tasks).

### 6.7 Inter-Step Cache Results

Table 7 shows the inter-step cache performance across all 8 real prompts (Q4_K_M, `entropy_exit`, $B$=256, $T$=16, 12 threads, seed=42).

Table 7: Inter-step cache performance (Q4_K_M, entropy_exit, $B$=256, steps=16, threads=12).

| **Prompt** | **No-Cache** | **Cache** | **Speedup** | **Steps** |
|---|---|---|---|---|
| Capital of France? | 17.5 tok/s | **24.4** tok/s | 1.39× | 3 |
| Translate to French | 25.9 | **27.7** | 1.07× | 2 |
| Python `is_prime` | 3.2 | **4.9** | 1.52× | 16 |
| Poem about ocean | 3.2 | **5.3** | 1.63× | 16 |
| Why sky is blue | 3.3 | **12.0** | 3.66× | 16 |
| List the planets | 3.3 | **9.4** | 2.84× | 15 |
| $15 \times 23 =$? | 12.8 | **15.7** | 1.22× | 4 |
| Translate to Spanish | 7.6 | **22.9** | 3.03× | 7 |
| **Average** | **9.6** | **15.3** | **1.59×** | |

The cache provides 1.59× average speedup with a clear pattern: prompts requiring more denoising steps benefit more. Two-step prompts gain only 1.07× (the active set is nearly the full sequence), while 16-step prompts gain up to 3.66× (most positions are cached after early steps).

**Regularization effect.** In 3 of 8 prompts (sky explanation, planet listing, Spanish translation), the cached output is qualitatively better than the uncached output. We attribute this to implicit regularization: reusing stable K,V from previous steps prevents the bidirectional attention mechanism from amplifying accumulated errors during the iterative denoising process. This phenomenon warrants further investigation but suggests that the cache approximation may be beneficial rather than merely tolerable.

**Fidelity tuning.** The `--cache-keep-active N` parameter extends the active set to include positions that changed within the last $N$ steps. At $N$=5, token fidelity (exact match with uncached output) reaches 59.4% with 1.64× speedup—a useful trade-off when output reproducibility matters.

### 6.8 Multi-Model Comparison: Dream-7B vs LLaDA-8B

Table 8 compares the two supported architectures on all 8 prompts under identical conditions (Q4_K_M, `entropy_exit` + inter-step cache, 12 threads, seed=42). Dream generates $B$=64 tokens; LLaDA generates $B$=256.

**Complementary strengths.** The two models exhibit complementary performance profiles. Dream-7B excels at *math and code*: it correctly solves $15 \times 23 = 345$ in 2 steps (21.6 tok/s), where LLaDA requires all 16 steps and produces an incorrect answer. Dream also generates a complete, correct `is_prime()` function in 7 steps (8.2 tok/s) versus LLaDA's 15 steps (4.5 tok/s). LLaDA-8B excels at *translation*: French translation converges in 2 steps at 25.7 tok/s (3.0× llama.cpp), while

Table 8: Dream-7B vs LLaDA-8B (Q4_K_M, entropy_exit + cache, 12 threads). Bold indicates the faster model per prompt.

| | Dream-7B | | LLaDA-8B | | vs llama.cpp | |
|---|---|---|---|---|---|---|
| **Prompt** | **tok/s** | **Steps** | **tok/s** | **Steps** | **Dream** | **LLaDA** |
| Capital of France? | **21.6** | 2 | 22.4 | 3 | 2.5× | 2.6× |
| Translate to French | 14.3 | 6 | **25.7** | 2 | 1.7× | 3.0× |
| $15 \times 23 =$? | **21.6** | 2 | 6.0 | 16 | 2.5× | 0.7× |
| Translate to Spanish | 13.2 | 10 | **23.3** | 5 | 1.6× | 2.7× |
| Python `is_prime` | **8.2** | 7 | 4.5 | 15 | 1.0× | 0.5× |
| Why sky is blue | 4.9 | 16 | **5.0** | 16 | 0.6× | 0.6× |
| List the planets | 4.9 | 16 | **9.5** | 16 | 0.6× | 1.1× |
| Poem about ocean | 4.5 | 16 | **5.0** | 16 | 0.5× | 0.6× |
| **Average** | **11.6** | | **12.7** | | 1.4× | 1.5× |

Dream requires 6 steps (14.3 tok/s). Both models struggle with creative writing (4.5–5.0 tok/s), requiring all 16 steps.

**Architecture effects.** Dream's GQA (28/4 heads) reduces KV computation by 7× compared to LLaDA's MHA (32/32 heads), but this advantage is partially offset by Dream's smaller generation buffer ($B$=64 vs $B$=256). The inter-step cache benefits both architectures; for Dream, cached K,V are stored post-GQA-expansion (at full $n_{\text{head}}$ dimensionality) to avoid recomputing the grouped repetition on each step.

**Practical recommendation.** Users should select the model based on their primary use case: Dream for math, code, and factual queries; LLaDA for translation and conversational tasks. Both outperform `llama.cpp` on 5–6 of 8 prompts.

## 7 Limitations

We are transparent about the limitations of this work.

**Model quality.** Both LLaDA-8B and Dream-7B produce lower-quality outputs than their AR counterparts (Llama-3-8B, Qwen2.5-7B). LLaDA shows occasional repetitions and incomplete code; Dream produces higher-quality math and code but sometimes generates hallucinated content. The throughput comparison is between paradigms, not quality-equivalent models.

**Different models.** We compare dLLMs (LLaDA-8B, Dream-7B) against AR models (Llama-3-8B). These share similar parameter counts and backbone architectures but differ in training procedure, data, and generation mechanism. The comparison demonstrates the *computational* regime inversion, not a head-to-head model quality contest.

**Generation buffer asymmetry.** Dream benchmarks use $B$=64 tokens while LLaDA uses $B$=256. This reflects practical constraints (Dream's Qwen2.5 tokenizer produces shorter outputs) but means the per-token throughput comparison is not perfectly controlled. Dream would likely achieve higher throughput at $B$=256.

**Prior art on scheduling.** Entropy-based scheduling is not novel. Prophet [5], EAGS [4], and Fast-dLLM [11] proposed related mechanisms. Our contribution is the implementation in a CPU-optimized engine and the demonstration that adaptive scheduling is particularly impactful in the compute-bound regime.

**Single hardware platform.** All benchmarks were conducted on a single AMD EPYC system. Generalization to Intel, ARM, Apple Silicon, or consumer-grade CPUs requires further validation. The regime inversion argument is architecture-independent, but the crossover point varies with the bandwidth-to-compute ratio.

**Prompt coverage.** Real-prompt evaluation used 8 prompts. While the PyTorch benchmark covered 42 prompts in 12 categories, comprehensive evaluation across thousands of diverse prompts remains future work.

**Cache approximation.** The inter-step KV cache reuses K,V tensors computed from stale hidden states. While empirically this does not degrade quality (and sometimes improves it), the approximation error grows with the number of steps between cache refresh. For very long generation sequences or models with stronger inter-position coupling, the cache may require periodic full refreshes.

## 8 Implications and Future Work

**Democratizing inference.** If diffusion models perform better on commodity CPUs, the implications are significant. A $200 server CPU with 32–64 cores could provide competitive LLM inference without any GPU. This eliminates the hardware supply chain bottleneck and enables deployment in resource-constrained environments.

**AR-to-diffusion conversion.** Recent work shows that AR models can be converted to diffusion models via continued pre-training with masking [7]. We estimate that LoRA fine-tuning for this conversion could cost as little as $250 in compute, creating a practical pipeline: take any AR model, fine-tune for diffusion, convert to GGUF, and deploy on CPU with `diffuse-cpp`.

**Multi-hardware validation.** A community hackathon or Kaggle competition could validate the regime inversion across diverse hardware: Intel Xeon, AMD Threadripper, Apple M-series, ARM server chips. We predict that the advantage of diffusion will increase with core count and decrease with memory bandwidth.

**Engine optimizations.** The inter-step KV cache (Section 4.1) demonstrates that significant speedups are achievable through caching strategies. Further optimizations remain unexplored: bidirectional Flash Attention [1] adapted for non-causal masks, delta attention (recomputing only changed hidden state contributions), batched inference for serving workloads, generation buffers beyond 256 tokens, and support for additional architectures (SEDD, MDLM). Dream-7B's GQA architecture also opens the possibility of a more memory-efficient inter-step cache that stores K,V at the reduced KV-head dimensionality rather than post-expansion.

**Better diffusion models.** The regime inversion becomes practically relevant when dLLM quality matches AR models. Investment in larger, better-trained diffusion models (analogous to the Llama series) would enable the computational advantages demonstrated here to translate into real-world deployment wins.

## 9 Conclusion

The field has assumed that diffusion language models are inherently slower than autoregressive models for inference. This assumption is correct on GPUs, where massive memory bandwidth compensates for AR's sequential memory access pattern. On commodity CPUs, however, the computational regime inverts: AR inference hits a hard bandwidth ceiling while diffusion inference scales with available compute.

We demonstrated this empirically with `diffuse-cpp`, the first C++ inference engine for diffusion LLMs, supporting two architectures: LLaDA-8B (Llama/MHA) and Dream-7B (Qwen2.5/GQA). On an AMD EPYC 12-core CPU, thread scaling reaches 7.4× for diffusion versus 2.4× for AR. With Q4_K_M quantization, entropy-exit scheduling, and inter-step KV cache, `diffuse-cpp` achieves 14–28 tok/s on easy prompts—up to 3.3× faster than `llama.cpp`'s 8.51 tok/s on the same hardware. The two models exhibit complementary strengths: Dream excels at math and code (21.6 tok/s), while LLaDA excels at translation (27.7 tok/s). The inter-step cache provides 1.6× average speedup with no quality degradation.

The contribution is not a claim that diffusion is universally faster. It is the identification and empirical validation of a hardware-dependent regime inversion, and the open-source tools to exploit it. As CPU core counts increase and diffusion model quality improves, this regime may become the dominant path to GPU-free LLM inference.

The code, models, benchmark data, and this paper are publicly available under open licenses.

## Acknowledgments

This work was inspired by the GGML library and llama.cpp [3]. The entropy-exit scheduler builds on ideas from Prophet [5] and the entropy-adaptive scheduling literature [4, 11]. Dream support was developed using the architecture specifications from [12]. Figures 1–3 were generated with Matplotlib.

## References

[1] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[2] Georgi Gerganov. GGML: Tensor library for machine learning. https://github.com/ggerganov/ggml, 2023.

[3] Georgi Gerganov et al. llama.cpp. https://github.com/ggerganov/llama.cpp, 2023.

[4] Hyukhun Koh, Minha Jhang, Dohyung Kim, Sangmook Lee, and Kyomin Jung. PLM-based discrete diffusion language models with entropy-adaptive Gibbs sampling. *arXiv preprint arXiv:2411.06438*, 2024.

[5] Pengxiang Li, Yefan Zhou, Dilxat Muhtar, Lu Yin, Shilin Yan, Li Shen, Soroush Vosoughi, and Shiwei Liu. Diffusion language models know the answer before decoding. *arXiv preprint arXiv:2508.19982*, 2025.

[6] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *International Conference on Machine Learning (ICML)*, 2024.

[7] Shen Nie, Fengqi Zhu, Chao You, Xiaojun Zhang, and Zhenguo Ou. LLaDA: Large language diffusion with mAsking. *arXiv preprint arXiv:2502.09992*, 2025.

[8] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 2024.

[9] Claude E Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.

[10] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[11] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025.

[12] Jiacheng Ye, Shansan Gong, Liheng Chen, Lin Zheng, Mukai Li, Thong Nguyen, Kaiyan Zhang, Shuicheng Yan, Silvio Savarese, Caiming Xiong, and Yingbo Zhou. Dream 7B – Scalable generation of structured and unstructured content from diffusion language modeling. *arXiv preprint arXiv:2504.16915*, 2025.

# A Regime Inversion Diagram

The $2 \times 2$ matching matrix illustrates the regime inversion:

| | **GPU** (High Bandwidth) | **CPU** (Low Bandwidth) |
|---|---|---|
| **Autoregressive** (BW-hungry) | MATCH<br>High BW feeds sequential reads | MISMATCH<br>Low BW bottlenecks sequential read |
| **Diffusion** (Compute-hungry) | REDUNDANT<br>Excess BW wasted on parallel compute | MATCH<br>Parallel compute fills CPU cores |